We covered four big ideas this week:

1. There is a universal TM, i.e., one that can simulate any other TM. This is not surprising since computers do this already.
2. Techniques to show undecidability: the first was a 'diagonalisation' argument, and the second was a 'reduction' argument.
3. Time complexity of an algorithm: this is a function that maps n to the largest number of steps the algorithm takes on any input of length n. This allowed us to define **P**, the languages that can be decided by polynomial-time TMs.
4. Nondeterministic TMs: such a machine accepts input u if some branch accepts, and rejects u otherwise. This allowed us to define **NP**, the languages that can be decided by polynomial-time NTMs.

After this tutorial you should be able to:

1. Show that certain languages are undecidable.
2. Compute the asymptotic time-complexity of a TM.
3. Show that certain languages are in **P**
4. Show that certain languages are in **NP**.

Nondeterministic TMs

Problem 1. Fix $\Sigma = \{0, 1\}$.

Give a low-level description of a nondeterministic TM N that decides the language

$$\{10^{n_1}10^{n_2}1 \cdots 10^{n_k} : n_i = n_j \text{ for some } i \neq j\}$$

E.g., 10001<u>00</u>1010<u>00</u>10000 should be accepted, but 10001001010000 should not.

Solution 1. Here is an implementation-level description: The NTM makes a left to right pass, nondeterministically marking two of the segments, say 10^{n_i} and 10^{n_j} (e.g., it can delete all the bits in the unchosen segments). It then uses a deterministic algorithm to check if $n_i = n_j$ (similar to the TM for $\{0^n1^n : n \geq 0\}$).

Time complexity of TMs

Problem 2. Calculate the time-complexity of the TMs TM2 (give your answer in big-Oh notation):

Solution 2. $\Theta(n^2)$. On all strings of length n the TM makes at most $n/2$ passes, each pass scanning $n/2$ cells (the ones between the leftmost 0 and the leftmost

1). This shows that the time complexity is $O(n^2)$. On strings in the language $\{0^n 1^n : n \geq 1\}$ it makes exactly $n/2$ passes; this shows that the time complexity is $\Theta(n^2)$.

Problem 3. Show that **P** is closed under union.

Solution 3. The proof in class that the decidable languages are closed under intersection also shows that that **P** languages are closed under union. Indeed, if M_1, M_2 have polynomial-time complexity, say $p_1(n)$ and $p_2(n)$, then the machine for their union that runs M_1, then runs M_2, and accepts if either of the machines accepted, and rejects otherwise, runs in time $p_1(n) + p_2(n)$, which is also a polynomial.

Problem 4. Consider the language $L_{\text{composite}}$ of decimal strings x whose numeric values are not prime. E.g., the following strings are in the language $4, 6, 10, 14, 25, 49, 292$, and the following strings are not $3, 7, 11, 97$.

Show that $L_{\text{composite}} \in$ **NP** by giving a high-level description and an implementation-level description of a NTM for the language that runs in polynomial time.

Solution 4. We give a high-level description of a nondeterministic decider for $L_{\text{composite}}$.

On input x:

1. nondeterministically guess y with $1 < y < x$.
2. test if y divides x. If yes, then accept, else reject.

A slightly more implementation-level description could do the following. Suppose numbers are encoding least-significant-digit first.

1. On a second tape, write a sequence of digits, and stop no later than the length of x (the TM knows when to stop because it can scan the input x on the first tape at the same time). The choice of digit at each step is chosen nondeterminitically.
2. Let y be the value written on the second tape when it stops writing.
3. If $x \geq y$ then reject.
4. Test if y divides x. If yes, then accept, else reject.

The number of steps the machine takes before producing y is $O(|x|)$. Checking if y divides x can be done in polynomial time in $|x|$ (Euclid's Algorithm!).

What happens if you try make a similar construction to try show that the complement of $L_{\text{composite}}$, i.e., the set of prime numbers, is in **NP**?

Undecidability

Problem 5. Show that the halting problem is undecidable. I.e., that the language

$$L_{\text{TM-Halting}} := \{\langle M, w\rangle \mid M \text{ is a TM that halts on input } w\}$$

is not decidable.

You can either do this directly by a self-referencing argument, or you can suppose that the problem is decidable and use this to build a decider for the TM-acceptance problem, which we know is undecidable. Give the second approach.

Solution 5. We give the second approach.

1. Suppose H decides this language, i.e., $H(\langle M, w\rangle)$ accepts if M halts on w, and rejects otherwise.
2. We build a TM D to decide $L_{\text{TM-acceptance}}$, which is a contradiction.
3. TM D takes $\langle M, w\rangle$ as input:
   (a) It runs $H(\langle M, w\rangle)$ and looks at whether it accepts or rejects.
   (b) If H rejects, then D rejects.
   (c) If H accepts, then D runs M on input w and waits for it to halt (which it will!)
   (d) If M accepts then D accepts, and if M rejects then D rejects.

Problem 6. Let L be the set of strings Source_M, x such that every instruction carried out during the run of M on x moves the head (left or right).

Show that L is undecidable.

Hint. Suppose L were decidable, take a TM for it, and build a TM for an undecidable problem.

Solution 6. We assume L is decidable by a TM N, and use this assumption to construct a decider for the acceptance problem for TMs (which we know has no decider, and thus conclude that our assumption is incorrect).

We give a high-level description of a decider $A(M, x)$ for the acceptance problem for TMs, as follows:

1. Convert M into an equivalent must-move TM M'.
2. Convert M' into an equivalent TM M'' by renaming `halt-accept` to `penultimate-accept` and renaming `halt-reject` to `penultimate-reject`, and adding the following 'do not move' transitions

```
penultimate-accept  *  *  *  halt-accept
penultimate-reject  *  *  *  halt-reject
```

(Note: If N accepts (M'', x) then M'' diverges on x; if N rejects (M'', x) then M'' halts on x.)

3. Run N on input (M'', x).
4. If N accepts then return reject (explanation: if N accepts (M'', x) then M'' diverges on x, and so x is not in $L(M'') = L(M)$).
5. If N rejects then simulate M'' on x and return what it does (explanation: if N rejects (M'', x) then M'' halts on x and so we may simulate it until it halts; if the simulation ends in `halt-accept` then x is in $L(M'') = L(M)$ and so we accept, and if it ends in `halt-reject` then x is not in $L(M'') = L(M)$ and so we reject).

Additional problems

Problem 7. Let $L_{\text{TM-regular}}$ be the set of strings Source_M such that $L(M)$ is regular.

Show that this language is undecidable.

Hint. Assume it is decidable, and let D be a decider for it. Write an algorithm that uses D and solves the acceptance problem for TMs.

Solution 7. A solution is in Sipser, Theorem 5.3, copied in Figure 1.

Problem 8. Calculate the time-complexity of the TMs TM1 (give your answer in big-Oh notation).

Solution 8. $\Theta(n)$. On all strings of length n the TM makes a single scan from left to right, and halts by the time it reaches the end of the string. This shows that the time complexity is $O(n)$. On strings in $L(a^*b^*)$ it only halts at the end of the string, which shows that the time complexity is $\Theta(n)$.

Problem 9. Prove that the equivalence problem for TMs is not decidable.

Hint: Suppose it were decidable; take a decider D for it; and build a decider for the acceptance problem for TMs.

Solution 9. Let M_{all} be some fixed TM with $L(M_{all}) = \Sigma^*$. For instance, M_{all} could simply ignore its input and go immediately into the accepting state.

Let D be a decider for the equivalence problem for TMs. We build another decider E that works as follows. It takes $\langle M, w \rangle$ as input, writes $\langle M_w, M_{all} \rangle$ on a second tape and runs D on that tape, and if D accepts then E accepts, and if D rejects then E rejects.

We now show that E decides the acceptance problem for TMs, which is a contradiction since we know (Turing) that no such decider exists.

THEOREM 5.3

$REGULAR_{\mathsf{TM}}$ is undecidable.

PROOF IDEA As usual for undecidability theorems, this proof is by reduction from A_{TM}. We assume that $REGULAR_{\mathsf{TM}}$ is decidable by a TM R and use this assumption to construct a TM S that decides A_{TM}. Less obvious now is how to use R's ability to assist S in its task. Nonetheless, we can do so.

The idea is for S to take its input $\langle M, w\rangle$ and modify M so that the resulting TM recognizes a regular language if and only if M accepts w. We call the modified machine M_2. We design M_2 to recognize the nonregular language $\{0^n1^n \mid n \geq 0\}$ if M does not accept w, and to recognize the regular language Σ^* if M accepts w. We must specify how S can construct such an M_2 from M and w. Here, M_2 works by automatically accepting all strings in $\{0^n1^n \mid n \geq 0\}$. In addition, if M accepts w, M_2 accepts all other strings.

Note that the TM M_2 is *not* constructed for the purposes of actually running it on some input—a common confusion. We construct M_2 only for the purpose of feeding its description into the decider for $REGULAR_{\mathsf{TM}}$ that we have assumed to exist. Once this decider returns its answer, we can use it to obtain the answer to whether M accepts w. Thus, we can decide A_{TM}, a contradiction.

PROOF We let R be a TM that decides $REGULAR_{\mathsf{TM}}$ and construct TM S to decide A_{TM}. Then S works in the following manner.

$S =$ "On input $\langle M, w\rangle$, where M is a TM and w is a string:

1. Construct the following TM M_2.
   $M_2 =$ "On input x:
   1. If x has the form 0^n1^n, *accept*.
   2. If x does not have this form, run M on input w and *accept* if M accepts w."
2. Run R on input $\langle M_2\rangle$.
3. If R accepts, *accept*; if R rejects, *reject*."

Figure 1: Proof that $L_{\text{TM-regular}}$ is undecidable

First, E is a decider since D is a decider. Second, we show that E accepts $\langle M, w\rangle$ iff M accepts w. Indeed, E accepts $\langle M, w\rangle$ if and only if D accepts $\langle M_w, M_{all}\rangle$ if and only if $L(M_w) = \Sigma^*$ if and only if M accepts w (by the second hint).

Problem 10. Define the non-acceptance problem for TMs:

$$L_{\text{TM-non-acceptance}} = \{\langle M, w\rangle \mid M \text{ is a TM that does not accept } w\}$$

Here is a proof that it is not recognisable.

Proof.

1. Suppose it were recognisable.
2. The complement of $L_{\text{TM-NonAcc}}$ is $L_{\text{TM-Acc}}$.
3. But we know that $L_{\text{TM-Acc}}$ is recognisable (easy).
4. So $L_{\text{TM-Acc}}$ is decidable (since both it and its complement are recognisable).
5. But we know this is not the case (Turing).

□

There is a small technical problem with this proof. What is it and how can it be fixed?

Hint: What is the complement of the language $L_{\text{TM-non-acceptance}}$?

Solution 10. The TM-acceptance problem is

$$L_{\text{TM-acceptance}} = \{\langle M, w\rangle \mid M \text{ is a TM that accepts } w\}$$

The complement of $L_{\text{TM-acceptance}}$ is not $L_{\text{TM-not-acceptance}}$. The reason is that a string is not in $L_{\text{TM-acceptance}}$ means that either it is of the form $\langle M, w\rangle$ and the TM M does not accept w, or it is not of the form $\langle M, w\rangle$ for any TM M. So really, the complement of $L_{\text{TM-acceptance}}$ is

$$L_{\text{TM-not-acceptance}} \cup E$$

where E is the language of strings that are not of the form $\langle M, w\rangle$. Note that E is decidable (we can easily tell if a string is not in the TM format we use in class, or if it has a syntax error).

Here, then, is the full argument that shows that $L_{\text{TM-non-acceptance}}$ is not recognisable.

1. Assume $L_{\text{TM-non-acceptance}}$ is recognisable.
2. We will show that the acceptance problem for TMs is decidable (which is a contradiction, see lecture slides where we show it is not).
3. We know the acceptance problem for TMs is recognisable (see lecture slides).

4. The complement of the acceptance problem for TMs is

$$L_{\text{TM-non-acceptance}} \cup E$$

   where E is defined above.

5. But then this language is recognisable: a TM takes a string as input, checks if it is in E, if not, so the input is in the form $\langle M, w \rangle$, runs the recogniser for the TM-non-acceptance problem on it. (More generally, the union of a recognisable language and a decidable language is recognisable).

6. Thus both the acceptance problem for TMs and the non-acceptance problem for TMs are recognisable, which by the Theorem in lectures means it is decidable (which is what we wanted to prove to get a contradiction).

Problem 11. Show that there is a decision problem that can't be solved by any Python program. Do not use the Church-Turing Thesis.

Hint. Formalise the acceptance problem for Python programs, and show that there is no Python program that decides it.

Solution 11. Let L be the set of strings $\langle P, w \rangle$ where P is a Python program that, on input w, halts and returns `true`.

To show that L is not decidable by a Python program, we can use the exact same argument that showed that the acceptance problem for TMs is not decidable. Suppose there is a Python program A that does decide it. Build a new Python program B that takes P as input, runs A on input $\langle P, \langle P \rangle \rangle$, waits for it to halt, and returns the opposite answer. Then ask what happens when we run B on input $\langle B \rangle$ to get a contradiction.

Problem 12. Show the following languages are in **P**:

1. $L(a^*b^*)$
2. $\{w\#w^R : w \in \{a, b\}^*\}$, where w^R is the reverse of w.
3. $\{ww : w \in \{a, b\}^*\}$

(Aside: which of these languages is not regular?)

Solution 12.

1. The first language is regular, and every regular language is decidable in linear time. To see this, note that a DFA can be viewed as a TM that, at every step, reads the next symbol and moves one cell to the right, and changes state just as the DFA does. When the TM reads a blank symbol, it accepts/rejects according to whether the DFA is in a final/non-final state.

2. Here is an implementation-level description: check the first unmarked letter is equal to the last unmarked letter, mark these, and repeat, until the only unmarked letter is #. This decider runs in quadratic time.
3. We give a high-level description/algorithm. On input $x = x_1 x_2 \cdots x_n \in \Sigma^*$:
   (a) Let $n = len(x)$.
   (b) If n is odd then reject.
   (c) Let $mid = n/2$.
   (d) Let $L = x[1, mid]$ and $R = x[mid + 1, n]$.
   (e) If $L = R$ then accept, else reject.
   The complexity of this algorithm is $\Theta(n)$ since to split the string into two would take linear time, as would checking equality between two strings.

Problem 13. In this problem you will show that the 3COL problem is in **NP**. The 3COL problem takes an undirected graph $G = (V, E)$ as input, and decides whether or not G has a proper 3-colouring, i.e., a partition of V set into three sets such that no edge of G has both its endpoints in the same set. The language corresponding to this problem is

$$L_{3\text{col}} := \{\langle G \rangle : G \text{ has a proper 3-colouring}\}$$

Show that the $L_{3\text{col}}$ is in **NP** by giving a high-level description of a NTM that decides it.

Also, give an implementation-level description.

Solution 13. Here is a high-level description of a nondeterministic decider for $L_{3\text{col}}$.

Take $\langle (V, E) \rangle$ as input:

1. Nondeterministically pick three subsets R, G, B of vertices that partition V.
2. For every edge (u, v) in E, check if both u and v are in the same set.
3. If none is, then accept, else reject.

Problem 14. A language is in **EXP** (aka *deterministic exponential time*) means that it is decidable by a TM with time-complexity $t(n) = 2^{n^k}$ for some k. Show that **NP** $\subseteq$ **EXP**. In words, this says that every **NP** problem is decidable in exponential time.

Solution 14. This follows from Theorem 7.8 and Theorem 7.11 in Sipser.

Theorem 7.8 says that for DTMs, one can simulate multiple tapes by a single tape at a cost of a quadratic blowup, i.e., if a DTM with multiple tapes has time

complexity $t(n)$ then there is an equivalent DTM with one tape that runs in time $O(t(n)^2)$. This can be seen by analysing the construction we gave in class for simulating a multitape machine by a single tape machine. See Sipser for details.

Theorem 7.11 says that for a given NTM, let b be the maximum number of nondeterministic choices given N's transition function. Then a computation tree on an input of size n is b-branching (i.e., every node has at most b children), and has depth at most $t(n)$. Thus, the size of the tree is at most $b^{t(n)} = 2^{O(t(n))}$, and since a BFS takes linear time in the size of the given tree/graph, the TM D runs in time $2^{O(t(n))}$.

So, if $f(n)$ is a polynomial, then D runs in exponential time. Note that here we are using the convention that an *exponential* function is of the form $b^{p(n)}$ for a polynomial $p(n)$ and constant b. Thus, e.g., 3^{n^2} is exponential, as is 3^n and 3^{2n}.

Both of these constructions assume that $t(n) \geq n$.